

HDOC Day-24

Increasing Patterns Using Nested Loops and Switch-Case in C

Aim

To develop a menu-driven C program using a switch statement, nested loops and separate void functions to print five different increasing patterns without using arrays.

Theory / How the Program Works

The program repeatedly displays a menu using a **do-while loop**. The user's choice is processed using **switch-case**. Each pattern is implemented in a separate function with no return value. Nested **for loops** are used: the outer loop controls the rows and the inner loop controls how many symbols, numbers or letters are printed in each row. Input validation ensures that the number of rows is positive; for the alphabet pattern, n is restricted to 1 through 26.

Menu

Choice	Pattern
1	Increasing right-angled triangle (*)
2	Increasing row-number triangle
3	Increasing natural-number pattern
4	Consecutive-number triangle
5	Increasing alphabet pattern
6	Exit

Algorithm

1. Start and declare choice.
2. Display the pattern menu and read the user's choice.
3. Use switch-case to call the function corresponding to choices 1-5; choice 6 exits and default reports an invalid choice.
4. Inside each pattern function, read n and validate it. For patterns 1-4, n must be positive; for the alphabet pattern, $1 \leq n \leq 26$.
5. For the star triangle, for each row i print i asterisks.
6. For the row-number triangle, for each row i print the number i exactly i times.
7. For the natural-number pattern, in row i print integers 1 through i.
8. For the consecutive-number triangle, initialize num=1 and keep increasing it after every printed value.
9. For the alphabet pattern, in row i print letters A through the ith letter.
10. After a pattern finishes, return to the menu. Repeat until Exit is selected.
11. Stop.

Test-Case Table

Test	Choice / Input	Expected result
1	1, n=4	Prints *, **, ***, **** in increasing rows.
2	2, n=4	Prints 1; 2 2; 3 3 3; 4 4 4 4.
3	3, n=4	Rows contain 1 through the corresponding row number.

Test	Choice / Input	Expected result
4	4, n=4	Prints consecutive values: 1 / 2 3 / 4 5 6 / 7 8 9 10.
5	5, n=4	Prints A / A B / A B C / A B C D.
6	5, n=27 then 4	Rejects 27 and accepts valid n=4.
7	1, n=0 then 3	Rejects 0 and accepts positive n=3.
8	choice=9	Displays invalid-choice message and menu repeats.
9	choice=6	Program exits.

C Program

```
#include <stdio.h>

void starTriangle(void);
void rowNumberTriangle(void);
void naturalNumberPattern(void);
void consecutiveTriangle(void);
void alphabetPattern(void);

int main()
{
    int choice;

    do
    {
        printf("\n===== INCREASING PATTERNS MENU =====\n");
        printf("1. Increasing right-angled triangle\n");
        printf("2. Increasing row-number triangle\n");
        printf("3. Increasing natural-number pattern\n");
        printf("4. Consecutive-number triangle\n");
        printf("5. Increasing alphabet pattern\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: starTriangle(); break;
            case 2: rowNumberTriangle(); break;
            case 3: naturalNumberPattern(); break;
            case 4: consecutiveTriangle(); break;
            case 5: alphabetPattern(); break;
            case 6: printf("Exiting program...\n"); break;
            default: printf("Invalid choice! Enter 1 to 6.\n");
        }
    } while (choice != 6);

    return 0;
}

void starTriangle(void)
{
    int n, i, j;
    do {
        printf("Enter number of rows: ");
        scanf("%d", &n);
        if (n <= 0)
            printf("Invalid input! Rows must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
}

void rowNumberTriangle(void)
{
    int n, i, j;
    do {
        printf("Enter number of rows: ");
        scanf("%d", &n);
        if (n <= 0)
            printf("Invalid input! Rows must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", i);
        printf("\n");
    }
}

void naturalNumberPattern(void)
{
    int n, i, j;
    do {
        printf("Enter number of rows: ");
```

```

        scanf("%d", &n);
        if (n <= 0)
            printf("Invalid input! Rows must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
}

void consecutiveTriangle(void)
{
    int n, i, j, num = 1;
    do {
        printf("Enter number of rows: ");
        scanf("%d", &n);
        if (n <= 0)
            printf("Invalid input! Rows must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", num++);
        printf("\n");
    }
}

void alphabetPattern(void)
{
    int n, i, j;
    do {
        printf("Enter number of rows (1-26): ");
        scanf("%d", &n);
        if (n < 1 || n > 26)
            printf("Invalid input! Enter rows from 1 to 26.\n");
    } while (n < 1 || n > 26);

    for (i = 1; i <= n; i++)
    {
        for (j = 0; j < i; j++)
            printf("%c ", 'A' + j);
        printf("\n");
    }
}

```

Compilation and Execution

The program was compiled using **gcc -Wall -Wextra hdoc_day24.c -o hdoc_day24**. Compilation completed successfully. It was executed with $n = 4$ for all five patterns and was also tested with an invalid menu choice.

Sample Output

```
===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Enter number of rows: *
* *
* * *
* * * *

===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Enter number of rows: 1
2 2
3 3 3
4 4 4 4

===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Enter number of rows: 1
1 2
1 2 3
1 2 3 4

===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Enter number of rows: 1
2 3
4 5 6
7 8 9 10

===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Enter number of rows (1-26): A
A B
A B C
A B C D

===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Invalid choice! Enter 1 to 6.

===== INCREASING PATTERNS MENU =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: Exiting program...
```

Result

The menu-driven C program was successfully written, compiled and executed. All five increasing patterns are produced correctly using nested loops and separate void functions, invalid row values are validated, invalid menu choices are handled, and the menu repeats until Exit is selected.